

API SECURITY RISK ANALYSIS REPORT

Public API Security Assessment

Target APIs:

JSONPlaceholder | ReqRes

Prepared By:

Asna Haris | Cybersecurity Intern
Future Interns

Date: 28 March 2026 | Classification: Internal

1. Executive Summary

This report presents the findings of an API security risk assessment conducted on two widely used public/demo APIs: JSONPlaceholder (<https://jsonplaceholder.typicode.com>) and ReqRes (<https://reqres.in>). The assessment was performed as part of Cybersecurity internship at Future Interns, following OWASP API Security Top 10 guidelines and standard penetration testing methodology using Postman and browser DevTools.

The analysis identified 8 security findings across both APIs, ranging from critical authentication weaknesses to informational observations. While these APIs are intentionally open for testing purposes, the risks identified directly mirror real-world vulnerabilities found in production SaaS environments. Each finding is documented with evidence, business impact, and remediation guidance.

Total Findings	8
Critical	2
High	2
Medium	2
Low	1
Info	1

2. Assessment Scope & Methodology

2.1 APIs Tested

API Name	Base URL	Purpose
JSONPlaceholder	https://jsonplaceholder.typicode.com	Fake REST API for testing
ReqRes	https://reqres.in	API testing & prototyping platform

2.2 Methodology

The assessment followed a structured process aligned with OWASP API Security Top 10 (2023 edition):

- Reconnaissance: Review API documentation and enumerate available endpoints
- Authentication Analysis: Test login, token handling, and access control
- Endpoint Testing: Send crafted requests using Postman to identify improper responses
- Input Validation Testing: Inject unexpected inputs (strings, integers, empty fields)
- Response Analysis: Inspect headers, status codes, and response bodies for data leakage
- Risk Classification: Assign CVSS-aligned severity ratings per OWASP categories
- Remediation Mapping: Provide actionable fixes for each identified risk

2.3 Tools Used

- Postman – API endpoint testing and request crafting
- Browser DevTools – Network traffic inspection and header analysis
- OWASP API Security Top 10 (github.com/OWASP/API-Security) – Risk framework
- JSONPlaceholder & ReqRes – Target demo APIs

3. Risk Summary

The table below summarizes all identified security findings. Detailed analysis for each finding follows in Section 4.

#	Risk / Finding	API / Endpoint	Severity	OWASP Category
1	No Authentication on Any Endpoint	JSONPlaceholder – ALL	Critical	API2:2023 – Broken Authentication
2	Unrestricted HTTP Methods (PUT/DELETE)	JSONPlaceholder – /posts/{id}	Critical	API5:2023 – Broken Function Level Auth
3	No Rate Limiting	Both APIs – ALL endpoints	High	API4:2023 – Unrestricted Resource Consumption
4	Sensitive Data Exposure in Responses	ReqRes – /api/users	High	API3:2023 – Broken Object Property Level Auth
5	Missing Security Headers	Both APIs – ALL	Medium	API8:2023 – Security Misconfiguration
6	Excessive Data Exposure	JSONPlaceholder – /users	Medium	API3:2023 – Broken Object Property Level Auth
7	Predictable / Sequential Resource IDs	JSONPlaceholder – /posts	Low	API1:2023 – Broken Object Level Auth
8	No HTTPS Enforcement / HSTS Header	JSONPlaceholder – ALL	Info	API8:2023 – Security Misconfiguration

4. Detailed Findings

Finding 1: No Authentication Required on Any Endpoint

Severity	Critical
API Tested	JSONPlaceholder (https://jsonplaceholder.typicode.com)
Endpoint	GET /users, GET /posts, GET /comments, DELETE /posts/1, etc.
OWASP Ref	API2:2023 – Broken Authentication

Description

JSONPlaceholder exposes all endpoints without any form of authentication. Any anonymous user on the internet can read, create, update, or delete any resource without providing credentials, tokens, or API keys. There is no login mechanism, session management, or identity verification of any kind.

Business Impact

In a real production environment modeled on this architecture, an attacker could: read all user data, delete critical records, impersonate users, and exfiltrate sensitive business information — all without any credentials. This represents a complete authentication bypass, classified as the most severe tier in OWASP API Security.

Evidence / Test Observation

Test performed: Sent a DELETE request to <https://jsonplaceholder.typicode.com/posts/1> with no Authorization header. API responded with HTTP 200 OK and returned the deleted object. No authentication was challenged at any point. Repeated across GET, POST, PUT, PATCH endpoints — all returned 200 with no auth required.

Remediation

Implement token-based authentication (e.g., OAuth 2.0 / JWT Bearer tokens) on all endpoints. Require clients to include an Authorization header. Validate tokens server-side on every request. Return HTTP 401 Unauthorized for unauthenticated requests. Apply the principle of least privilege — even authenticated users should only access their own resources.

Finding 2: Unrestricted HTTP Methods Allow Unauthorized Write Operations

Severity	Critical
API Tested	JSONPlaceholder (https://jsonplaceholder.typicode.com)
Endpoint	PUT /posts/{id}, DELETE /posts/{id}, PATCH /posts/{id}
OWASP Ref	API5:2023 – Broken Function Level Authorization

Description

The API accepts and processes all HTTP methods (GET, POST, PUT, PATCH, DELETE) on every endpoint without any authorization or role-based access control. A non-authenticated user can modify or delete any resource by simply changing the HTTP method. This breaks function-level authorization — the server does not verify whether the caller has permission to perform write operations.

Business Impact

Without function-level authorization, any external party can delete or modify records as if they were an admin. In a production SaaS system this would allow: data destruction, content manipulation,

competitor sabotage, and potential regulatory violations (GDPR data integrity requirements). The business impact is severe: data loss, reputational damage, and legal liability.

Evidence / Test Observation

Test performed: Sent PUT `https://jsonplaceholder.typicode.com/posts/1` with a JSON body `{ title: 'Hacked', body: 'Modified', userId: 1 }`. Response: HTTP 200 OK with the modified object returned — confirming the server accepted an unauthorized write operation. No token or session required.

Remediation

Restrict HTTP methods per endpoint. Use an allowlist — only permit the exact methods each endpoint needs. Implement role-based access control (RBAC): GET for readers, POST/PUT/DELETE only for authenticated owners or admins. Return HTTP 403 Forbidden for unauthorized method attempts. Log all write operations for auditing.

Finding 3: No Rate Limiting — Vulnerable to Brute Force and DoS

Severity	High
API Tested	JSONPlaceholder & ReqRes
Endpoint	All endpoints
OWASP Ref	API4:2023 – Unrestricted Resource Consumption

Description

Neither API implements rate limiting or throttling. An attacker can send an unlimited number of requests per second without being blocked or slowed down. This enables brute-force attacks on login endpoints, credential stuffing, and denial-of-service (DoS) by overwhelming the server with requests.

Business Impact

For a production API, absence of rate limiting allows: brute-forcing user credentials (login endpoints), scraping entire user databases, overwhelming backend infrastructure (DoS/DDoS), and bypassing business logic that relies on per-user request quotas (e.g., free-tier limits). Cloud hosting costs can also spike unexpectedly under abuse.

Evidence / Test Observation

Test performed on ReqRes: Sent 200 consecutive POST requests to `https://reqres.in/api/login` with varying credentials using Postman's Collection Runner. All 200 requests were processed with no throttling, no CAPTCHA challenge, no delay, and no HTTP 429 Too Many Requests response at any point.

Remediation

Implement rate limiting at the API gateway level (e.g., max 100 requests/minute per IP). Return HTTP 429 Too Many Requests with a `Retry-After` header when limits are exceeded. Use exponential backoff for repeated failures on auth endpoints. Consider CAPTCHA for login after 5 failed attempts. Use tools like AWS API Gateway throttling, Nginx rate limiting, or specialized services (e.g., Cloudflare).

Finding 4: Sensitive User Data Exposed in API Responses

Severity	High
API Tested	ReqRes (<code>https://reqres.in</code>)
Endpoint	GET <code>/api/users</code> , GET <code>/api/users/{id}</code>

Description

The ReqRes /api/users endpoint returns user profile data in its responses, including first name, last name, email address, and avatar URL. While this is demo data, the pattern mirrors a real-world vulnerability where APIs return more fields than necessary — including fields that should be restricted to the user themselves or to administrators.

Business Impact

If mirrored in a real SaaS product, this pattern enables: user enumeration (attackers can list all users), PII harvesting for phishing campaigns, targeted attacks using real names and emails, and compliance violations under GDPR (unlawful disclosure of personal data). Email harvesting from a user list endpoint is a common precursor to spear-phishing.

Evidence / Test Observation

Requested GET https://reqres.in/api/users without any authentication. Response included: { id, email, first_name, last_name, avatar } for all users in the system. No scope restriction or field filtering was applied. Any anonymous caller can enumerate all users and extract their personal information.

Remediation

Apply the principle of data minimization. Return only the fields the requesting user is authorized to see. Use field-level access control: public endpoints should return only non-sensitive fields (e.g., display name, avatar). Sensitive fields (email, phone) should only be returned to the authenticated account owner or admins. Implement response filtering at the API layer, not the client layer.

Finding 5: Missing Critical Security Headers

Severity	Medium
API Tested	JSONPlaceholder & ReqRes
Endpoint	All endpoints (HTTP Response Headers)
OWASP Ref	API8:2023 – Security Misconfiguration

Description

Inspection of HTTP response headers from both APIs reveals the absence of several security-critical headers. Missing headers include: Content-Security-Policy (CSP), X-Frame-Options, X-Content-Type-Options, Referrer-Policy, and Permissions-Policy. These headers provide defense-in-depth at the browser layer and protect against a range of client-side attacks.

Business Impact

Missing security headers expose applications consuming these APIs to: clickjacking attacks (missing X-Frame-Options), MIME-type sniffing vulnerabilities (missing X-Content-Type-Options), cross-site scripting via content injection (missing CSP), and information leakage via referrer headers. While server-side impact is limited, client applications relying on these APIs inherit the risk.

Evidence / Test Observation

Response headers inspected using Browser DevTools Network tab. Confirmed absence of: Content-Security-Policy, X-Frame-Options, X-Content-Type-Options: nosniff, Referrer-Policy, Permissions-Policy. Only basic headers (Content-Type, Access-Control-Allow-Origin) were present. Neither API returns HSTS headers.

Remediation

Add the following headers to all API responses via server configuration or middleware: X-Content-Type-Options: nosniff; X-Frame-Options: DENY; Referrer-Policy: no-referrer; Content-Security-Policy: default-src 'self'; Permissions-Policy: geolocation=(), microphone=(). Use a security headers scanner (securityheaders.com) to validate after implementation.

Finding 6: Excessive Data Exposure — Full User Objects Returned

Severity	Medium
API Tested	JSONPlaceholder (https://jsonplaceholder.typicode.com)
Endpoint	GET /users, GET /users/{id}
OWASP Ref	API3:2023 – Broken Object Property Level Authorization

Description

The /users endpoint returns full user objects containing: id, name, username, email, address (including geolocation coordinates), phone number, website, and company details. This level of detail is excessive for most use cases and constitutes a violation of the data minimization principle.

Business Impact

Returning geolocation, phone, email, and full address in a single unauthenticated API response enables: physical location tracking, targeted social engineering using real-looking data, building detailed user profiles for attacks, and violation of data protection regulations (GDPR Article 5 – data minimization). Aggregating such data at scale is trivial through automated scraping.

Evidence / Test Observation

GET https://jsonplaceholder.typicode.com/users returned objects with 10+ fields per user including: address.geo.lat, address.geo.lng, phone, email, company.name, company.catchPhrase. This data was returned to an anonymous request with no authentication. The response included 10 full user records in a single call.

Remediation

Implement field-level filtering. Define separate DTOs (Data Transfer Objects) for public and private contexts. Public user profiles should return only: id, name, username. Authenticated users should see their own profile data. Admin-only endpoints should expose full records with appropriate authorization. Never return geographic coordinates, phone numbers, or company details in public endpoints.

Finding 7: Predictable Sequential Resource IDs Enable BOLA

Severity	Low
API Tested	JSONPlaceholder (https://jsonplaceholder.typicode.com)
Endpoint	GET /posts/{id}, GET /users/{id}, GET /comments/{id}
OWASP Ref	API1:2023 – Broken Object Level Authorization (BOLA)

Description

All resources in JSONPlaceholder use simple sequential integer IDs (1, 2, 3...). This predictable pattern makes it trivial for an attacker to enumerate and access any resource by simply incrementing the ID parameter. In a production system with authorization, this would enable Broken Object Level Authorization (BOLA) — the #1 OWASP API vulnerability.

Business Impact

Predictable IDs allow attackers to: iterate through all user records, access other users' private data by guessing IDs, verify the total count of records in the system, and perform automated data exfiltration. BOLA is the most commonly exploited API vulnerability in real-world breaches, including high-profile incidents at major companies.

Evidence / Test Observation

Manually tested: GET /users/1 through /users/10 all returned valid user records. Confirmed that IDs are sequential integers starting at 1. An attacker can enumerate all users by simply incrementing the ID parameter — no special knowledge or privilege required.

Remediation

Replace sequential integer IDs with UUIDs (Universally Unique Identifiers) or other non-guessable identifiers (e.g., nanoid). Even with random IDs, always enforce object-level authorization: verify that the requesting user is authorized to access the specific resource before returning it. Return HTTP 403 Forbidden (not 404) when an authenticated user requests another user's resource.

Finding 8: No HTTPS Enforcement / Missing HSTS Header

Severity	Info
API Tested	JSONPlaceholder (https://jsonplaceholder.typicode.com)
Endpoint	All endpoints
OWASP Ref	API8:2023 – Security Misconfiguration

Description

While JSONPlaceholder does support HTTPS, it does not return an HTTP Strict Transport Security (HSTS) header in responses. HSTS instructs browsers to always use HTTPS for future connections, preventing SSL-stripping attacks. Additionally, HTTP requests to the API are not automatically redirected to HTTPS in all configurations.

Business Impact

Without HSTS, users connecting for the first time over HTTP are vulnerable to man-in-the-middle (MITM) attacks where a network attacker downgrades the connection to HTTP (SSL stripping) and intercepts all traffic including tokens and sensitive data. This is particularly relevant on public Wi-Fi networks.

Evidence / Test Observation

Inspected response headers for Strict-Transport-Security — header was absent from all responses. Confirmed HTTPS is available but not enforced with HSTS. A client that initially connects via HTTP will not be automatically upgraded to HTTPS on subsequent connections.

Remediation

Add the HSTS header to all HTTPS responses: Strict-Transport-Security: max-age=31536000; includeSubDomains; preload. Redirect all HTTP traffic (port 80) to HTTPS (port 443) with HTTP 301 Moved Permanently. Submit the domain to the HSTS preload list (hstspreload.org) for browser-level enforcement.

5. OWASP API Security Top 10 Coverage

This assessment was conducted using the OWASP API Security Top 10 (2023) as the primary risk framework. The reference repository is available at: <https://github.com/OWASP/API-Security>

OWASP Category	Finding(s)	Severity
API1 – Broken Object Level Auth	Finding 7: Predictable Sequential IDs	Low
API2 – Broken Authentication	Finding 1: No Auth on Any Endpoint	Critical
API3 – Broken Object Property Level Auth	Findings 4, 6: Data Exposure & Excessive Fields	High / Medium
API4 – Unrestricted Resource Consumption	Finding 3: No Rate Limiting	High
API5 – Broken Function Level Auth	Finding 2: Unrestricted HTTP Methods	Critical
API8 – Security Misconfiguration	Findings 5, 8: Missing Headers & HSTS	Medium / Info

6. Remediation Roadmap

The following prioritized action plan is recommended for development teams. Items are ordered by risk severity and implementation effort.

Pri	Action	Timeline	Effort	Finding
1	Implement JWT / OAuth 2.0 authentication on all endpoints	Immediate (Sprint 1)	High	#1
2	Add RBAC – restrict write methods to authorized roles	Immediate (Sprint 1)	High	#2
3	Configure API Gateway rate limiting (100 req/min/IP)	Short-term (Sprint 2)	Medium	#3
4	Apply response field filtering / minimize exposed PII	Short-term (Sprint 2)	Medium	#4,6
5	Add all missing security response headers via middleware	Short-term (Sprint 2)	Low	#5
6	Replace sequential IDs with UUIDs across all resources	Medium-term (Q2)	Medium	#7
7	Enforce HTTPS with HSTS header and HTTP-to-HTTPS redirects	Short-term (Sprint 2)	Low	#8

7. Conclusion

This API Security Risk Analysis identified 8 findings across JSONPlaceholder and ReqRes, two widely used public testing APIs. While these platforms are intentionally open and unsecured for demonstration purposes, the vulnerabilities observed directly map to real-world risks that affect production SaaS systems daily.

The most critical findings — complete lack of authentication and unrestricted HTTP method access — represent foundational security failures that would be catastrophic in any production environment. These findings align with the top categories in the OWASP API Security Top 10, which documents the most exploited API vulnerabilities seen in real-world breaches at major companies.

Key takeaways for development teams and security professionals:

- Authentication is non-negotiable. Every write endpoint must require a valid, verified token.
- Authorization must be enforced server-side — never trust client-supplied identity.
- Rate limiting is a business continuity control, not just a security control.
- Data minimization reduces both breach impact and regulatory risk.
- Security headers are free to implement and provide significant defense-in-depth.

All remediation guidance in this report aligns with OWASP API Security best practices. Developers are encouraged to review the OWASP API Security Project at <https://github.com/OWASP/API-Security> for comprehensive implementation guidance.

8. References

- OWASP API Security Top 10 (2023): <https://github.com/OWASP/API-Security>
- JSONPlaceholder API: <https://jsonplaceholder.typicode.com>
- ReqRes API: <https://reqres.in>
- OWASP Top 10 Web Application Risks: <https://owasp.org/www-project-top-ten/>
- Security Headers Reference: <https://securityheaders.com>
- HSTS Preload List: <https://hstspreload.org>
- NIST Cybersecurity Framework: <https://www.nist.gov/cyberframework>

9. Disclaimer

This assessment was conducted solely for educational purposes as part of the Future Interns Cybersecurity Internship program. All testing was performed against public demo APIs specifically designed for learning and testing. No real user data was accessed or stored. No production systems were targeted. All findings are presented for educational awareness only.